

16.COMPARE DIFFERENT TYPES CLASSIFICATION ALGORITHMS AND EVALUATE THEIR PERFORMANCE.

1. Student Result Classification – Decision Tree

Study Hours	Attendance	Result
2	60	Fail
3	65	Fail
4	70	Fail
5	75	Pass
6	80	Pass
7	85	Pass
8	90	Pass
9	95	Pass

PROGRAM

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
df = pd.read_csv("student.csv")
X = df[["Study Hours", "Attendance"]]
y = df["Result"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
model = DecisionTreeClassifier(random_state=1)
```

```
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Predicted:", pred)
print("Actual:", y_test.values)
print("Accuracy:", accuracy_score(y_test, pred))
```

OUTPUT

Predicted: ['Pass' 'Fail' 'Pass']

Actual: ['Pass' 'Fail' 'Pass']

Accuracy: 1.0

Accuracy = 100%

2. Loan Approval – KNN

Income	Credit Score	Loan
8	780	Yes
7	760	Yes
6	720	Yes
4	620	No
3	580	No
9	800	Yes
5	650	No
8	770	Yes
4	600	No
7	740	Yes

PROGRAM

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
df = pd.read_csv("loan.csv")
X = df[["Income", "Credit Score"]]
y = df["Loan"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Predicted:", pred)
print("Actual:", y_test.values)
print("Accuracy:", accuracy_score(y_test, pred))
```

OUTPUT

Predicted: ['No' 'Yes' 'Yes']

Actual: ['No' 'Yes' 'Yes']

Accuracy: 1.0

Accuracy = 100%

3. Disease Classification – Naïve Bayes

Temperature	Heart Rate	Disease
39.0	110	Positive
38.5	108	Positive
37.0	78	Negative
39.2	115	Positive
36.8	75	Negative
38.8	112	Positive
37.2	80	Negative
39.1	114	Positive
36.7	74	Negative
38.9	111	Positive

PROGRAM

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
df = pd.read_csv("disease.csv")
X = df[["Temperature", "Heart Rate"]]
y = df["Disease"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
model = GaussianNB()
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Predicted:", pred)

```

```
print("Actual:", y_test.values)
print("Accuracy:", accuracy_score(y_test, pred))
```

OUTPUT

Predicted: ['Negative' 'Positive' 'Positive']

Actual: ['Negative' 'Positive' 'Positive']

Accuracy: 1.0

Accuracy = 100%

4. Employee Promotion – Logistic Regression

					Performance	Promotion
Experience						
10					95	Yes
8					90	Yes
7					88	Yes
3					65	No
2					60	No
9					93	Yes
4					70	No
8					89	Yes
2					58	No
6					85	Yes

PROGRAM

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

```

from sklearn.metrics import accuracy_score
df = pd.read_csv("employee.csv")
X = df[["Experience", "Performance"]]
y = df["Promotion"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Predicted:", pred)
print("Actual:", y_test.values)
print("Accuracy:", accuracy_score(y_test, pred))

```

OUTPUT

Predicted: ['No' 'Yes' 'Yes']

Actual: ['No' 'Yes' 'Yes']

Accuracy: 1.0

Accuracy = 100%

5. Fruit Classification – Random Forest

Weight	Sweetness	Fruit
150	90	Apple
160	88	Apple
140	91	Apple
120	70	Orange
125	72	Orange
130	74	Orange

155	89	Apple
118	69	Orange
148	92	Apple
122	71	Orange

PROGRAM

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
df = pd.read_csv("fruit.csv")
X = df[["Weight", "Sweetness"]]
y = df["Fruit"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
model = RandomForestClassifier(
    n_estimators=20, random_state=1)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Predicted:", pred)
print("Actual:", y_test.values)
print("Accuracy:", accuracy_score(y_test, pred))
```

OUTPUT

```
Predicted: ['Apple' 'Apple' 'Orange']
Actual:    ['Apple' 'Apple' 'Orange']
Accuracy: 1.0
```